AUTHORITATIVE ARCHITECTURE BRIEF

End-to-End Correlation in Azure n-Tier and Microservices Architectures

W3C Trace Context, OpenTelemetry, Azure API Management, Java JDK 25 on App Service, messaging causality, Azure Database for PostgreSQL, Azure Managed Redis, Microsoft Entra ID, and Azure Monitor / Application Insights

Purpose. Expert architecture review and implementation handoff. This document consolidates the substantive positions and disputes explicitly identified in the research request, reconciles material claims against standards and official Microsoft sources, and converts the result into atomic execution and certification criteria.

Unredacted technical content, redacted security material. No credentials, connection strings, access tokens, personal data, or tenant-specific secrets are embedded.

Document control and evidence method

Field	Value
Document status	Architecture decision brief; implementation-ready baseline
Environment	Browser SPA / server-rendered App Service → Azure API Management → Java JDK 25 App Service microservices → PostgreSQL and cache; synchronous and asynchronous branches
Primary correlation standard	W3C Trace Context with OpenTelemetry span semantics
Preferred backend	Workspace-based Azure Monitor Application Insights and Log Analytics
Evidence hierarchy	1) W3C / OpenTelemetry specifications; 2) Microsoft Learn / Azure product documentation and REST schemas; 3) official Microsoft GitHub repositories or issues; 4) field reports; 5) explicit inference
Claim labels	Verified; Partially verified; Unsupported; Incorrect; Outdated
Important reconstruction note	The indexed prior-chat retrieval returned no content. Therefore the report preserves every prior position and disagreement expressly enumerated in the request, but does not invent verbatim historical wording or attribute unstated details to another model.

A material claim is marked **Verified** only when authoritative evidence supports the behavior as stated. **Partially verified** means a narrower form is supported. **Unsupported** means no authoritative support was found for the specific statement. **Incorrect** conflicts with a specification or documented architecture. **Outdated** was once plausible but is no longer current as of the cutoff.

> **Temporal integrity**
> Product status is time-sensitive. The report records the state found at the 31 October 2026 cutoff. Preview features remain excluded from the production dependency path unless their preview terms, regional availability, and operational rollback are explicitly accepted.

Executive recommendation

Adopt W3C Trace Context as the only technical trace-causality protocol. Let the browser SDK create or continue the trace; configure APIM diagnostics for W3C; instrument each Java process with exactly one OpenTelemetry provider—preferably the supported Azure Monitor/Application Insights Java agent for this App Service architecture; and propagate context through HTTP and message metadata. A shared log field is useful, but is not a trace.

- **Do not mutate standards context.** Never rewrite a valid `traceparent` sampled flag from `00` to `01`. Sampling is a local recording/export decision; crossing a gateway does not justify falsifying upstream intent.
- **Do not derive W3C trace IDs from `context.RequestId`.** APIM request IDs and W3C trace IDs have different semantics and validation rules. If the edge must start a trace, use a standards-compliant tracer or allow the first instrumented service to create one.
- **Add a separate durable `business.invocation_id`** only when a business operation outlives a trace, is replayed, or spans multiple independent traces. Generate/validate it at a trusted edge and treat it as application data, not as a replacement trace ID.
- **Use links for many-to-one causality.** Fan-in, batches, and replay cannot be represented honestly with one arbitrary parent. Retain message context and create span links from the aggregation or replay span.
- **Use one workspace-based Application Insights resource per workload and environment by default.** Shared Log Analytics workspaces can still centralize governance and cross-resource queries. Split only for sovereignty, RBAC, ownership, retention, billing, or incompatible data-boundary requirements.
- **For new cache architecture, choose Azure Managed Redis.** Azure Cache for Redis Basic/Standard/Premium retires 30 September 2028 and Enterprise retires 30 March 2027; Azure Managed Redis is GA and is the service direction.[S22]
- **Certification is a 100%-fidelity controlled mode, not the permanent production sampling policy.** Prove ID invariants at full collection, then adopt a documented production sampling strategy with error/critical-path exceptions and cost controls.

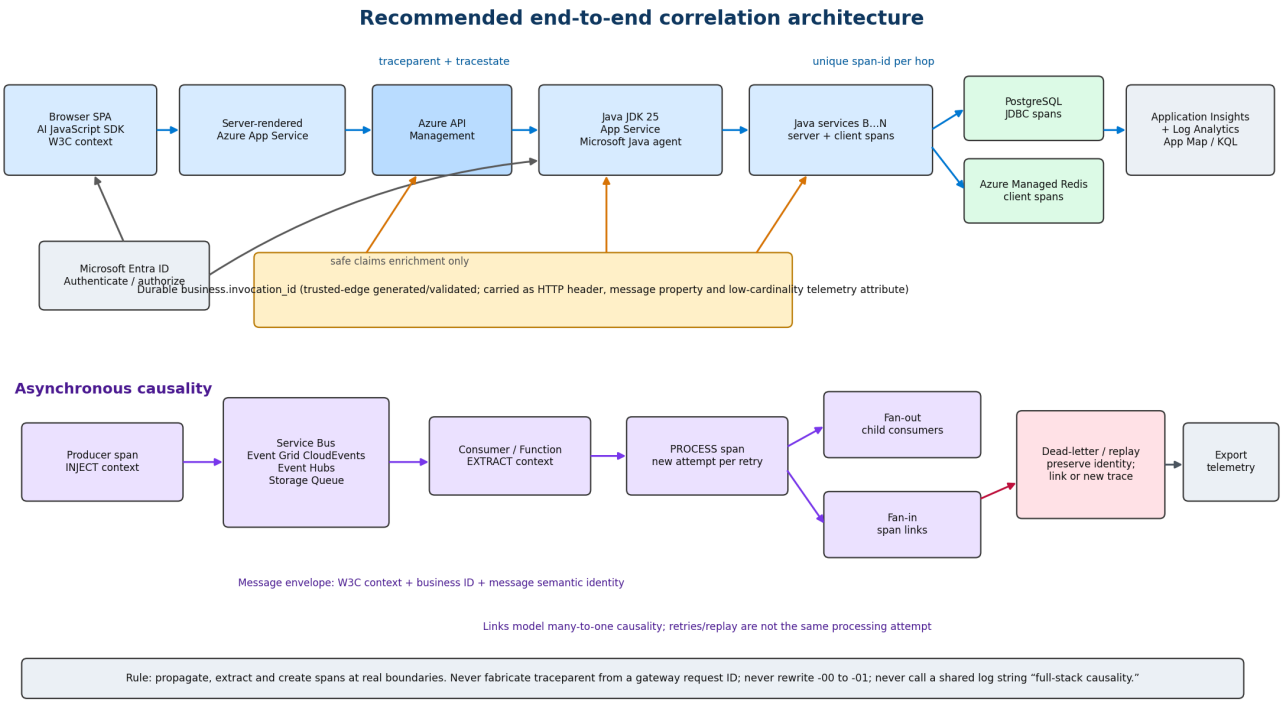


Figure 1. Corrected reference architecture and correlation boundaries.

1. Architecture principles and decision statements

ID	Principle	Decision
P1	Causality over coincidence	A trace is a graph of spans joined by W3C context and span relationships. Merely copying a GUID into logs provides search correlation but not timing, parentage, sampling state, or causal structure.
P2	Propagate; do not synthesize unconstrained context	Extract and validate inbound W3C context; create a new root only when no valid context exists or a trust boundary intentionally breaks it.
P3	One provider per process	Run either the Azure Monitor/Application Insights Java agent as the OpenTelemetry provider or a deliberate programmatic OpenTelemetry SDK configuration—not both. Duplicate providers/ exporters create double spans, conflicting IDs, and unpredictable suppression.
P4	Record boundaries that matter	Server spans represent accepted work; client/producer spans represent outbound operations; consumer/process spans represent message handling; links represent non-tree causality.
P5	Separate trace identity from business identity	Trace IDs are ephemeral technical causality. A business invocation ID is durable domain correlation with its own privacy, validation, lifetime, and cardinality controls.
P6	Gateway observability is not application observability	APIM diagnostics capture gateway request/dependency behavior but cannot replace service spans, message spans, database client spans, or application logs.
P7	Sampling layers are independent until proven otherwise	Browser, APIM, Java agent, Collector, and backend ingestion can each sample. The architecture must document every decision point and never assume one component obeys another component's diagnostic sampling configuration.
P8	Preview is an experiment, not a hidden production prerequisite	Azure Monitor Traces / Trace Analytics and native OTLP pathways may add value, but production causality must survive without preview-only components.

2. History of positions and competing opinions

The research contained a stable core recommendation plus challenge positions. Because the prior transcript was not present in the indexed context, this section preserves the explicit substance supplied in the request rather than fabricating exact quotations.

Position family	Substantive position	Disposition
Original recommendation	Use W3C Trace Context end to end; instrument browser, APIM, Java services and dependencies; export to Application Insights; add a durable business invocation ID when a trace cannot cover the business lifetime.	Retained with tighter trust, sampling, and async rules.
Gateway-centric opinion	Treat APIM as the central correlation authority, potentially generating traceparent from `context.RequestId`, correcting a `-00` flag to `-01`, and expecting APIM policy calls to materialize complete spans.	Rejected as a general architecture. APIM is a propagation and telemetry boundary, not the universal trace authority.
"Shared correlation ID" opinion	A single ID copied into headers and logs is sufficient for full-stack correlation.	Rejected for technical causality; retained only as optional business/search correlation.
Backend-centric opinion	Application Insights can reconstruct causality from telemetry after ingestion even if propagation or parentage is missing.	Rejected. Backend analytics cannot infer missing causal edges reliably.
Automatic-instrumentation-first opinion	Use the Azure Monitor Java agent to minimize code changes and cover HTTP/JDBC/Redis/common frameworks.	Retained, subject to JDK 25 compatibility testing, one-provider rule, and custom spans for unsupported frameworks/ business boundaries.

Programmatic OTel opinion	Use the OpenTelemetry SDK/Collector for portable configuration, tail sampling and multi-backend routing.	Retained as an alternative, not combined casually with the Java agent. Collector benefits must be evidenced and operationally justified.
APIM native OTLP opinion	APIM emits native OTLP spans and can be treated as an OTel SDK participant.	Rejected/unsupported. Documented APIM integration is Application Insights diagnostics; do not design around native OTLP emission without current product evidence.
.NET messaging opinion transplanted to Java	Service Bus package 7.13.0 and `Azure.Experimental.EnableActivitySource` are material requirements.	Removed as irrelevant to this Java architecture; they are .NET-oriented claims.
Classic adaptive sampling opinion	The Java path uses Application Insights classic adaptive sampling semantics.	Corrected. Use the Java agent / Azure Monitor OTel sampling controls documented for the pinned version; do not transpose classic SDK behavior.
One AI resource for everything	Put every environment and workload in one Application Insights resource for easy correlation.	Amended. Default one resource per workload/environment, with shared workspace where governance requires it.

3. Claim-by-claim reconciliation matrix

Claim	Class	Evidence rationale	Architecture impact
APIM supports W3C diagnostic correlation	Verified	APIM diagnostic resources expose W3C correlation configuration.	Set `httpCorrelationProtocol` to `W3C`; verify emitted IDs in a test trace.
APIM always emits `traceparent …-00`	Unsupported	No authoritative evidence establishes this as a universal current behavior. A sampled flag of 00 is valid W3C context, not inherently an APIM defect.	Test the deployed gateway/runtime/SKU; do not construct a workaround as an architectural dependency.
Rewrite `-00` to `-01` at APIM	Incorrect	Trace flags are part of propagated context. Rewriting records a false upstream sampling decision and can produce inconsistent partial traces.	Remove rewrite policies. Decide local recording/ export independently.
Build traceparent from `context.RequestId`	Incorrect	Request ID and W3C trace-id semantics differ; string formatting cannot guarantee compliant randomness, uniqueness, or trust.	Never use as the default trace root algorithm.
APIM emits native OTLP spans	Unsupported	Documented gateway integration targets Application Insights diagnostics, not a general OTLP SDK/exporter contract.	Use Application Insights gateway telemetry; evaluate OTLP elsewhere.
`send-request`/`wait` automatically yields complete child spans	Partially verified	Policy calls are observable gateway operations, but complete OTel-style internal spans and exact parentage are not a portable policy guarantee.	Validate in transaction diagnostics; add explicit application telemetry where causality matters.
APIM sampling controls Java agent sampling	Incorrect	Gateway diagnostics and service instrumentation are separate telemetry producers.	Document separate rates and certification mode.
APIM payload logging has no material performance risk	Incorrect	Body/header logging increases work, data volume and privacy exposure; Microsoft guidance recommends minimizing payload capture.	Log only required headers/metadata; keep bodies disabled except bounded troubleshooting.
W3C trace-id maps to AI `operation_Id`	Verified	Application Insights correlation uses operation identity and parent identity fields to model distributed operations.	Certification must compare the entire trace-id across requests/dependencies and validate parent IDs.
A shared GUID in every log proves causality	Incorrect	It carries no parent-child graph, timestamps, span kind, or sampling semantics.	Treat as a searchable business attribute only.
Java agent can cover App Service JDK 25 without qualification	Partially verified	App Service runtime availability and agent compatibility are separate support claims; automatic instrumentation libraries may lag a new JDK.	Pin agent version and run startup, load and bytecode-instrumentation tests before certification.
Run Java agent plus a second global OTel SDK provider	Incorrect	Multiple providers/exporters can duplicate or conflict.	One provider per JVM; use agent extensions or manual spans through the active API.
JDBC spans are PostgreSQL server-native traces	Incorrect	JDBC instrumentation observes client calls. PostgreSQL platform logs/metrics are separate and usually do not share trace context.	Correlate by time/resource safely; do not claim server span continuity.
Redis client command key/value should be captured	Incorrect	Keys/values can contain sensitive data and create cardinality/volume risk.	Capture safe operation/system/server attributes; avoid raw keys, values, scripts and arguments.
Entra request/correlation IDs are trace IDs	Incorrect	Identity service diagnostic IDs have different scope and lifecycle.	Store safe identity request IDs as attributes only when needed; never overwrite W3C IDs.
Business invocation ID should always equal trace ID	Incorrect	Business operations can outlive, retry, replay or cross multiple traces.	Keep separate; optionally record trace-id ↔ invocation-id membership.
Span parentage models fan-in directly	Incorrect	A span has one parent; many-to-one relationships need links.	Use links for aggregation/batches/replay.

100% production sampling is required for “full correlation”	Partially verified	It is required for deterministic certification and selected forensic paths, but may be too costly for steady state.	Use bounded certification mode; choose a production strategy after measured cost and gap analysis.
Azure Cache for Redis is the default for a new design	Outdated	The service is retiring; Azure Managed Redis is GA and is the stated successor.	Target Azure Managed Redis; plan migration for existing caches.
Collector GA means every Azure native OTLP intake path is GA	Incorrect	The upstream Collector can be GA while an Azure receiver/AMA/AKS/native endpoint remains preview.	Track status per component and ingestion path.
One AI resource for all workloads/ environments is always best	Unsupported	Convenience does not override RBAC, retention, ownership, billing and data-boundary concerns.	Default one per workload/environment; centralize via workspace/cross-resource queries.

Disputed APIM claims—bottom line
Configure and test documented W3C diagnostics. Do not mutate trace flags, fabricate traceparent, assume native OTLP, or rely on policy control-flow constructs to produce complete application-level spans. Minimize logged bodies and headers.

4. W3C Trace Context and OpenTelemetry model

Element	Required meaning	Implementation note
`traceparent`	Version, 16-byte trace-id, 8-byte parent-id and trace flags. IDs of all zeros are invalid.	Forward a valid value; create a new root when absent/invalid or intentionally broken at a trust boundary.[S1]
`tracestate`	Vendor-specific list accompanying traceparent.	Preserve ordering and limits according to W3C; do not place secrets or user data in it.[S1]
trace-id	Identity of the distributed trace.	32 lowercase hexadecimal characters in W3C representation; maps to AI operation identity.
span-id	Identity of one operation/span.	16 hexadecimal characters; every real span boundary gets a new span-id.
sampled flag	Upstream trace flag indicating recorded/ sampled intent.	Advisory propagation signal; a component can make its own recording decision. Do not falsify it in transit.
Parent	One causal predecessor in a trace tree.	A span has at most one parent.
Link	Relationship to another span context without parenthood.	Required for fan-in, batches, replay and other many-to-one relationships.[S2]
Application Insights `operation_Id`	Distributed operation grouping.	Expected to carry the W3C trace-id for correlated telemetry.[S3]
Application Insights `id`	Telemetry item/span identity.	Expected to carry the current span ID in W3C-based instrumentation.
Application Insights `operation_ParentId`	Parent telemetry/span identity.	Must equal the actual parent span ID for a strict parent-child edge.

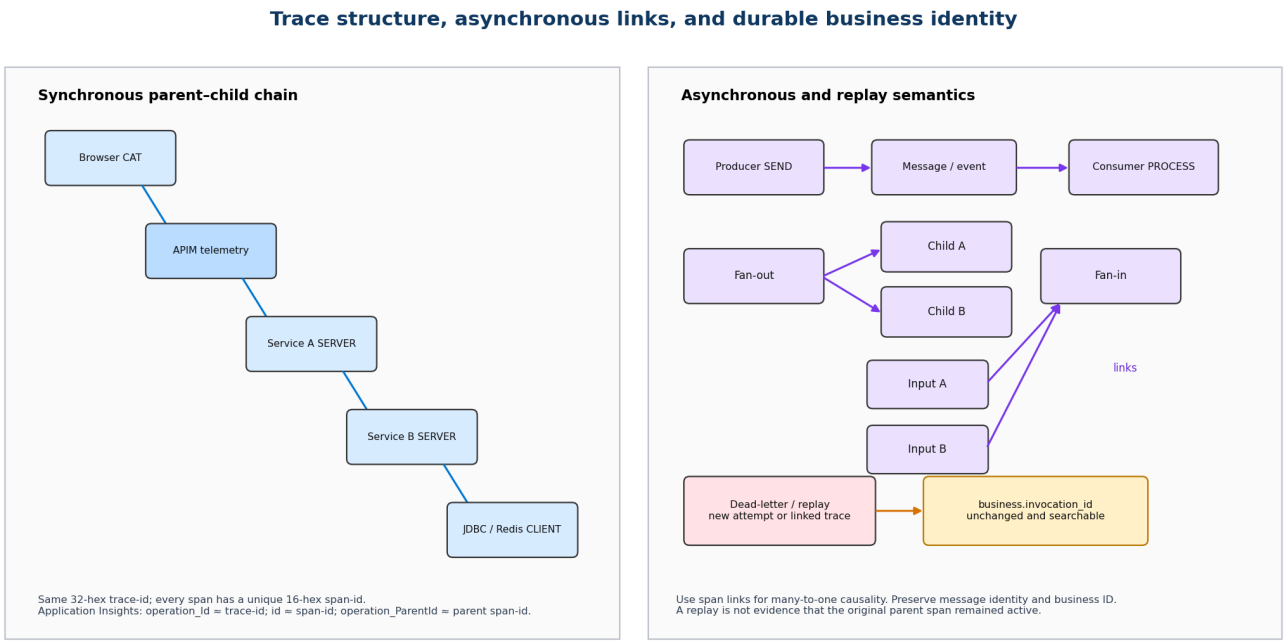


Figure 2. Synchronous parentage, asynchronous links, replay and durable business identity.

5. Browser and server-rendered front end

- Use the Application Insights JavaScript SDK for SPA real-user monitoring. Configure W3C distributed tracing mode and ensure the SDK is initialized before network calls that must be traced.[S12]

- For cross-origin AJAX/fetch, the browser must be permitted to send correlation headers. The target CORS policy must allow the relevant headers; the JavaScript SDK correlation-domain controls must include trusted API origins and exclude third-party domains that must not receive trace context.
- Instrument route changes and user interactions carefully: page views are not automatically identical to backend request spans. Validate the exact AJAX dependency to APIM and its server-side continuation.
- For server-rendered App Service, instrument the server process independently. The browser's page telemetry and the server render request may form different roots unless the navigation/request context is propagated and captured by supported instrumentation.
- User and session context are analytical dimensions, not proof of authorization. Do not collect tokens, email addresses, display names or raw object IDs. Prefer a keyed HMAC/pseudonymous stable identifier under documented retention and deletion controls.
- Keep correlation header exclusion lists for CDNs, analytics vendors, identity endpoints and other domains that should not receive internal context. Treat `tracestate` as potentially externally visible metadata.

6. Java JDK 25 on Azure App Service

Preferred implementation: run a pinned Azure Monitor/Application Insights Java agent release that is explicitly validated against the chosen JDK 25 App Service image. Automatic instrumentation should cover inbound HTTP, outbound HTTP and common JDBC/Redis libraries; unsupported frameworks and business boundaries require manual spans using the active OpenTelemetry API.[S7][S21]

Concern	Production guidance
Runtime versus agent support	Do not equate "App Service offers JDK 25" with "every agent/instrumentation library supports JDK 25." Record the exact App Service runtime image, Java agent version and instrumentation library versions tested.
Startup	Attach the agent once using the supported App Service startup/environment mechanism. Fail deployment if startup logs show transformation errors, duplicate providers or exporter rejection.
Configuration	Inject the Application Insights connection string as a secret setting/reference; never place it in source or the PDF. Pin configuration in source-controlled deployment artifacts.
Service identity	Set a stable `service.name` / Application Insights cloud role name per logical service and environment. Do not use instance IDs as role names. Instance identity belongs in `service.instance.id` or platform attributes.
Logging correlation	Use structured logging and the instrumentation-supported MDC/context fields for trace-id and span-id. Avoid hand-parsing traceparent. Verify IDs in emitted logs against current spans.
Sampling	Configure the sampler in the one active provider. Verify the pinned Java agent's current default rather than assuming classic adaptive sampling.
Manual spans	Use the OpenTelemetry API already supplied by the agent/provider. Never install a second global SDK solely to create custom spans.
One provider	Agent or programmatic SDK—one. A Collector is an exporter/router outside the process, not a second in-process provider.

```
# Illustrative startup pattern only; pin an approved artifact and validate against JDK 25.
JAVA_TOOL_OPTIONS="-javaagent:/home/site/wwwroot/agent/applicationinsights-agent.jar"
APPLICATIONINSIGHTS_CONNECTION_STRING="@Microsoft.KeyVault(SecretUri=...)"
OTEL_SERVICE_NAME="orders-api-prod"

# Do not add a second TracerProvider in application code when the agent owns the provider.
```

> **Compatibility gate**
> Certification requires a successful JDK 25 smoke/load test for HTTP server/client, JDBC, selected Redis client, logging MDC, messaging SDKs, shutdown flush and redeploy/restart behavior. A service that starts but silently loses spans is not compatible.

7. Azure API Management implementation

Control	Decision / implementation
Logger	Provision an APIM Application Insights logger pointing to the intended workspace-based Application Insights resource. Use an approved connection-string secret delivery pattern; if the selected APIM API version/SKU supports managed-identity authentication for the logger, validate it explicitly rather than assuming identity replaces the resource configuration.
Diagnostic resource	Create API- or all-APIs-scope diagnostics through IaC and set `httpCorrelationProtocol: W3C`. Record API version and exported resource state.[S4][S5]
Sampling	Set a deliberate gateway diagnostic sampling percentage. It controls APIM telemetry volume, not Java-agent sampling. Use 100% in certification.
alwaysLog	Use for defined exceptional categories such as errors where supported; verify interaction with sampling in the exact API version. Do not interpret it as an end-to-end “always sample” signal.
Headers	Allowlist only headers required to diagnose routing/correlation. Treat Authorization, cookies, tokens, PII and business secrets as forbidden.
Bodies	Default request/response body bytes to zero. Temporarily enable bounded capture only under controlled troubleshooting with customer-data handling approval.
Policy dependencies	Treat `send-request` and composite policy control flow as gateway implementation details. Verify what appears in Application Insights; create application-level spans where causal visibility is mandatory.
Inbound context	Forward valid traceparent/tracestate. At external trust boundaries, validate and optionally start a new trace according to a documented policy while recording the rejected value only in safe hashed form if needed.
What not to do	No `-00 → -01` rewrite; no trace-id synthesized from `context.RequestId`; no assumption of native OTLP; no broad payload logging; no belief that gateway diagnostics replace service instrumentation.

```
// Conceptual ARM/Bicep property shape; verify the exact current API schema.
properties: {
  loggerId: apimLoggerResourceId
  alwaysLog: 'allErrors'
  sampling: { samplingType: 'fixed', percentage: 100 }
  httpCorrelationProtocol: 'W3C'
  frontend: { request: { headers: ['x-business-invocation-id'], body: { bytes: 0 } } }
  backend:  { request: { headers: ['x-business-invocation-id'], body: { bytes: 0 } } }
}
```

The APIM pricing documentation notes that throughput depends on policy count/type, payload sizes, backend performance and other factors, and requires load testing rather than relying on generic throughput figures.[S6] This is the correct basis for treating extensive diagnostic/body logging as a measured performance and cost decision, not a harmless toggle.

8. Asynchronous propagation: Java-specific rules

Service	Java architecture rule
Service Bus	Inject W3C context into application properties through supported Azure SDK instrumentation or explicit propagator. Create producer/SEND and consumer/PROCESS spans. Preserve message ID, delivery count and dead-letter reason as safe attributes. Retries are new processing attempts; dead-letter/replay should retain business ID and link or start a new trace.
Event Grid CloudEvents	Do not overload CloudEvent `id` with trace-id. Use CloudEvents distributed-tracing extension attributes or transport metadata supported by the path; validate subscriber propagation. Event Grid delivery is at-least-once in common patterns, so duplicates and ordering must be handled.
Event Hubs	Model the send/receive/batch operation and partition safely. A batch can contain events from multiple contexts; one arbitrary parent is wrong. Create links to each contributing event context when useful and bounded.

Storage Queue	Queue messages lack a universal native trace envelope. Define versioned metadata in the message or a compact envelope; enforce size, validation and privacy. Functions bindings may not expose every property uniformly, so test trigger behavior.
Azure Functions / Java worker	Use the current supported Java/OpenTelemetry integration for the worker model. Verify trigger-created spans, extracted parent context, duplicate host/worker spans and log correlation. Preview examples for another language are evidence of a platform direction, not Java proof.

Retries, dead-letter and replay

- **Retry:** preserve the message's original propagated context but create a new processing attempt span. Record attempt count; do not reuse the prior span ID.
- **Dead-letter:** preserve technical context and business ID only if the transport/envelope safely supports them. Record dead-letter reason and original message identity; never store tokens or payload PII in span attributes.
- **Replay:** a delayed replay may intentionally start a new trace. Add a link to the original producing/processing context when available; keep `business.invocation_id` unchanged if it is the same business operation.
- **Fan-out:** each consumer creates its own PROCESS span from the propagated message context.
- **Fan-in:** aggregator span has one normal parent at most and links to all contributing contexts within link-count/ cardinality limits.
- **Batching:** retain per-message context in a bounded representation. Do not choose the first message as parent and imply the others are descendants.

> **.NET claims removed**
> Service Bus package `7.13.0` and `Azure.Experimental.EnableActivitySource` are not requirements for this Java design. If a Java SDK version gate exists, it must be sourced from the Java package release notes and validated independently.

9. PostgreSQL and Redis data-tier correlation

Azure Database for PostgreSQL

- Use supported Java JDBC instrumentation to emit CLIENT dependency spans around database calls. The span is produced in the application process and proves that the service attempted a database operation.[S20][S21]
- Capture safe attributes such as database system, logical database name where approved, server endpoint and sanitized operation name. Avoid raw parameter values. Treat full statements as sensitive and high-volume; prefer normalized operation or query fingerprint if governance permits.
- Client spans are not PostgreSQL server-native spans. Azure PostgreSQL platform metrics, slow-query logs and server logs have separate clocks, retention and schemas. They can support diagnosis but usually do not continue W3C parentage.
- Connection pooling means the client span must wrap the logical operation, not connection acquisition alone. Retries by drivers/frameworks can create multiple dependencies; make retry count visible without recording credentials or query parameters.
- If Microsoft Entra authentication is used for PostgreSQL, token acquisition is identity activity, not a database child span. Never log tokens or connection strings.[S20]

Redis and migration direction

- Instrument the actual Java client (for example Lettuce/Jedis/Redisson) only if the pinned Java agent instrumentation supports that version. Unsupported clients need manual CLIENT spans at a safe abstraction boundary.
- Record operation category, endpoint and outcome. Do not record raw cache keys, values, Lua scripts, command arguments or session identifiers. Hashing a highly unique key can still preserve cardinality and privacy risk.
- Cache hits and misses may be metrics or bounded span attributes; avoid a custom span for every internal cache helper if the client span already exists.
- **New architecture:** provision Azure Managed Redis. Azure Cache for Redis Basic/Standard/Premium retire 30 September 2028; Enterprise/Enterprise Flash retire 30 March 2027. Microsoft identifies Azure Managed Redis as GA and recommends migration.[S22]
- Existing Azure Cache for Redis deployments need a migration plan, compatibility testing, connection-endpoint cutover, failback plan, persistence/replication assessment and trace comparison before/after migration.

10. Microsoft Entra ID and safe identity enrichment

Microsoft Entra ID authenticates principals and issues tokens; the application and APIM enforce authorization. Identity telemetry enriches a trace but does not define its causality. Entra request IDs and correlation IDs are identity-service diagnostic identifiers and must never replace the W3C trace-id.

- Prefer role/group/tenant category attributes that are required for troubleshooting and have bounded cardinality. Do not record raw JWTs, refresh tokens, authorization headers or claims bundles.
- For user analysis, create a pseudonymous identifier using a secret-keyed HMAC over a stable internal subject, with key rotation and access controls. A plain hash of an email/object ID is usually reversible by enumeration.
- Record authentication outcome and policy category rather than token contents. Authorization failures should remain correlated to the request span.
- When contacting Entra support, retain Entra-specific request/correlation IDs in a separate security-approved field. Their presence is evidence of an identity request, not an edge in the application trace graph.

11. Durable business invocation ID

Question	Decision
When needed?	Long-running workflow exceeds trace retention/lifetime; retries or replays create new traces; human approval pauses the flow; multi-system business operation needs a durable searchable key.

Where created?	At the first trusted application edge after authentication/validation. External caller-supplied values are accepted only under explicit format, length, namespace and authorization rules.
Format	Opaque random identifier (for example UUIDv4/ULID under a documented standard). Do not encode customer/user/order data.
HTTP propagation	Dedicated allowlisted header such as `x-business-invocation-id`; validate length/charset and overwrite untrusted duplicates.
Messaging propagation	Dedicated application property or versioned envelope field; preserve across retry/dead-letter/replay when business semantics remain the same.
Telemetry	Attach as `business.invocation_id` to spans/logs/events required for lookup. Avoid turning it into a metric dimension.
Privacy/cardinality	Treat as pseudonymous/high-cardinality operational data. Restrict access, define retention, and never expose it as an authentication secret.
Relationship to traces	One business invocation can map to one or many traces. Technical trace IDs remain authoritative for parentage; the business ID is a secondary membership/search key.

12. Sampling and fidelity

Layer	Decision
Browser	Choose a documented SDK sampling rate; certification uses 100%. Browser sampling does not automatically command APIM or Java to record.
APIM	Diagnostic sampling is independent. Certification uses 100% and `alwaysLog` for defined errors if supported. Verify actual item presence.
Java agent / OTel SDK	Use the sampler of the single active provider. Recent Microsoft Java distributions/agents can use rate-limited or ratio-based controls; verify the pinned release's documented default and override. Do not label it "classic adaptive sampling" without evidence.
Collector	Tail sampling is available only when the Collector receives complete candidate traces and has enough memory/time to decide. It changes failure modes and must be load tested.
Ingestion/ backend	Workspace transformations, retention and ingestion limits can remove or alter searchable data after export. They are not trace sampling but affect reconstruction.

Certification mode

- 100% recording/export on browser, APIM and all participating Java/worker processes for a bounded test window and controlled synthetic traffic.
- No body logging and no sensitive-data relaxation. Full fidelity means all spans, not all payloads.
- Fixed test vectors that cover success, error, retry, timeout, fan-out, fan-in, batch, dead-letter and replay.
- Export and preserve evidence: raw traceparent values at trusted test points, Application Insights rows, query results, configuration snapshots and version inventory.

Production strategy

After certification, set cost-aware head/rate sampling based on measured request volume, troubleshooting objectives and regulatory requirements. Preserve errors/critical flows through supported controls or use a Collector tail-sampling design only when completeness, capacity and operational ownership are proven. Never promise 100% reconstructability of every production request when any layer samples.

13. Application Insights and Log Analytics topology

Default: one workspace-based Application Insights resource per workload and environment, connected to a governed Log Analytics workspace. This creates stable role naming and clean ownership while preserving cross-resource/ workspace query options.

Topology	Use when	Risk / exception
AI resource per workload + environment; shared LAW	Default enterprise pattern; separate app configuration, alerts and ownership with centralized workspace governance.	Workspace RBAC can be broad; use resource-context access and validate data boundaries.
Dedicated LAW per regulated boundary	Sovereignty, retention, legal hold, tenant or operational isolation requires separation.	Cross-workspace queries and unified operations become more complex.
Single AI resource for many services	Small, single-owned workload where role names are reliable and RBAC/retention are identical.	Noisy maps, accidental collisions, broad access and harder cost attribution.
Separate AI resource per microservice	Only when independent ownership, retention, billing or release isolation truly requires it.	Operational fragmentation and alert/query sprawl.

Application Map nodes are identified from role/name properties and edges from dependency telemetry; Transaction Diagnostics correlates server and dependency items into an end-to-end view.[S26][S27] These tools visualize emitted telemetry—they cannot reconstruct missing spans or corrected parentage.

14. GA, preview and ingestion landscape as of 31 October 2026

Capability	Status / classification	Architecture use
Azure Monitor/Application Insights Java agent	Production-supported path, subject to the pinned version's support matrix.	Primary direct-export option for Java App Service.
Workspace-based Application Insights, Application Map, Transaction Diagnostics, KQL	Production Azure Monitor capabilities.	System of record for this architecture.
OpenTelemetry Collector core	GA upstream software; deployment and exporter maturity are separate decisions.	Optional router/processor for portability, multi-backend export or tail sampling.
Azure Monitor direct/native OTLP ingestion	Treat each documented endpoint/path separately. Do not infer GA from Collector GA.	Adopt only if official current documentation marks the exact path GA for the target environment.
Azure Monitor Traces / Trace Analytics	Public Preview at the research cutoff unless the exact regional/product documentation states otherwise.	Evaluation lane only; no production certification dependency.
AMA/AKS OpenTelemetry collection paths	Some paths remain preview even though the Collector is GA.	Do not use App Service guidance interchangeably with AKS/AMA.
Azure Managed Redis	GA and recommended successor.	Default for new cache deployments.[S22]

Status rule
"Uses OpenTelemetry" is not a release-status statement. Record the status of the instrumentation library, exporter, receiver, hosting extension, ingestion endpoint and analytics experience separately.

15. Architecture alternatives

Option	Strengths	Costs / when not to choose	Recommendation
Azure-native direct export	Lowest moving parts; Java agent and APIM integrate directly with Application Insights; familiar Azure operations.	Primarily Azure backend; limited tail sampling / routing flexibility.	Default for this environment.
Azure-native + OpenTelemetry Collector	Central processors, routing, transformation, tail sampling, multi-backend export and egress control.	New highly available stateful-ish telemetry tier; queue/memory/ backpressure/security/upgrade burden; App Service networking considerations.	Use only for a proven requirement and after scale testing.
Third-party APM platform	Can provide unified multicloud operations, mature tail sampling, proprietary analytics or existing enterprise standards.	Additional data egress, cost, duplication, privacy and operational integration.	Select only when an evidenced gap justifies it; retain W3C/Otel portability.

16. Component responsibility matrix

Component	Context creation/ extraction	Propagation	Span creation	Export	Business ID	Auto vs custom
Browser SDK	Create/continue browser trace	Inject into trusted CORS calls	Browser/page/AJAX spans	Export browser telemetry	Attach safe pseudonymous context; propagate business ID if present	Mostly automatic; domain/privacy configuration custom
Server-rendered App Service	Create/continue server request	Inject outbound HTTP	Server + dependency spans	Java/server agent export	Generate business ID only if this is trusted first edge	Automatic plus custom edge validation
APIM	Validate/forward valid context; not universal root authority	Forward W3C + allowlisted business ID	Gateway request/ dependency diagnostics	APIM logger to AI	Validate/overwrite untrusted business header per policy	Configuration/policy; no invented traceparent
Java service	Extract inbound	Inject HTTP/ message context	SERVER, CLIENT, PRODUCER, custom spans	Single provider export	Attach/propagate invocation ID	Agent automatic + custom boundaries
PostgreSQL client	N/A	N/A	JDBC CLIENT dependency	Through Java provider	No raw business ID in query	Automatic if supported
Redis client	N/A	N/A	Redis CLIENT dependency	Through Java provider	No raw keys/values	Automatic if supported
Message producer	Use active span context	Inject into application properties/envelope	SEND/PRODUCER span	Through Java provider	Propagate business ID	SDK instrumentation or custom
Message consumer / Function	Extract message context	Inject on downstream calls	PROCESS/ CONSUMER span; links for batches	Worker/ provider export	Preserve across retry/ replay	Validate binding/worker behavior
Collector (optional)	Does not create app causality	Preserves context only in telemetry	May create internal telemetry about pipeline	Routes/ processes/ exports	Must not leak high-cardinality attributes	Platform team owned
Application Insights / LAW	No context creation	No propagation	Stores/visualizes received spans	N/A	Search / RBAC / retention	Backend responsibility

17. Detailed sequence flows

Synchronous flow

- 1. Browser SDK starts/continues a trace and creates a dependency span for the API call; it injects valid W3C headers only to an allowed APIM origin.

- 2. APIM receives the request, validates/forwards W3C context, records gateway telemetry through the Application Insights diagnostic, and forwards the validated `business.invocation_id`.
- 3. Java service A extracts context and creates a SERVER span with a new span-id under the inbound parent. Its structured logs carry the active trace-id/span-id.
- 4. Service A creates a CLIENT span for service B and injects its context. Service B creates a SERVER child span.
- 5. Service B creates separate CLIENT spans for JDBC and Redis calls. Database/cache services do not become W3C participants unless explicitly instrumented server-side; client spans remain the causal boundary.
- 6. Every exporter sends telemetry to the intended Application Insights resource. The backend maps operation identity/parentage and renders Application Map / Transaction Diagnostics.

Asynchronous flow

- 1. Producer creates a SEND/PRODUCER span, injects W3C context into the transport-safe message metadata, and carries `business.invocation_id`.
- 2. Broker/event service persists/routes the message. Do not assume it emits a child span unless documented telemetry exists.
- 3. Consumer extracts the message context and creates a PROCESS span for that delivery attempt. Logs and downstream calls use the active context.
- 4. On transient failure, the next delivery creates a new PROCESS span; record delivery/attempt count. Do not reuse span IDs.
- 5. On dead-letter/replay, preserve message and business identity. If original context is still valid and policy permits, link; otherwise start a new trace linked to the original context.
- 6. Fan-out produces independent child/process spans. Fan-in aggregation uses span links to all contributing contexts; it does not select a false single parent.

18. Verified implementation patterns and snippets

Java manual span through the active provider

```
// Use the OpenTelemetry API exposed by the one active provider (agent or SDK).
private static final Tracer tracer = GlobalOpenTelemetry.getTracer("com.example.orders", "1.0");

Span span = tracer.spanBuilder("orders.validate")
    .setSpanKind(SpanKind.INTERNAL)
    .startSpan();
try (Scope ignored = span.makeCurrent()) {
    span.setAttribute("business.invocation_id", validatedInvocationId);
    validateOrder();
} catch (Throwable t) {
    span.recordException(t);
    span.setStatus(StatusCode.ERROR);
    throw t;
} finally {
    span.end();
}
```

Trusted-edge business ID logic

```
1. Read x-business-invocation-id.
2. If caller is trusted AND value matches approved length/charset/namespace, accept it.
3. Otherwise generate a new opaque random ID and ignore/replace the inbound value.
4. Add the value to response header only if exposing it is approved.
5. Attach as business.invocation_id to the active span and structured logs.
6. Propagate as an allowlisted HTTP header and message property.
7. Never use it as an authentication or authorization credential.
```

Current KQL: reconstruct a trace

```
let traceId = "<32-hex-trace-id>";
union isfuzzy=true
    (AppRequests | project TimeGenerated, ItemType="request", Name, Id, ParentId, OperationId, AppRoleName,
Success, DurationMs, Properties),
    (AppDependencies | project TimeGenerated, ItemType="dependency", Name, Id, ParentId, OperationId,
AppRoleName, Success, DurationMs, Properties),
    (AppTraces | project TimeGenerated, ItemType="trace", Name=Message, Id=tostring(Properties["span_id"]),
ParentId=tostring(Properties["parent_span_id"]), OperationId, AppRoleName, Success=bool(null),
DurationMs=real(null), Properties),
    (AppExceptions | project TimeGenerated, ItemType="exception", Name=ExceptionType,
Id=tostring(Properties["span_id"]), ParentId=tostring(Properties["parent_span_id"]), OperationId,
AppRoleName, Success=false, DurationMs=real(null), Properties)
| where OperationId == traceId
| order by TimeGenerated asc
```

If the workspace uses legacy table names, use `requests`, `dependencies`, `traces` and `exceptions` with their legacy column names. Do not mix schemas without verifying the workspace.

Business invocation lookup

```
let invocationId = "<opaque-id>";
union isfuzzy=true AppRequests, AppDependencies, AppTraces, AppExceptions, AppEvents
| extend InvocationId = tostring(Properties["business.invocation_id"])
| where InvocationId == invocationId
| summarize FirstSeen=min(TimeGenerated), LastSeen=max(TimeGenerated), Items=count(),
            Roles=make_set(AppRoleName, 100), Traces=make_set(OperationId, 500)
            by InvocationId
```

Candidate orphan detection

```
let lookback = 2h;
let spans = union isfuzzy=true
    (AppRequests | where TimeGenerated > ago(lookback) | project TimeGenerated, Id, ParentId, OperationId,
AppRoleName, Name, Kind="request"),
    (AppDependencies | where TimeGenerated > ago(lookback) | project TimeGenerated, Id, ParentId,
```

```
OperationId, AppRoleName, Name, Kind="dependency");
let ids = spans | where isnotempty(Id) | summarize by OperationId, Id;
spans
| where isnotempty(ParentId)
| join kind=leftanti ids on OperationId, $left.ParentId == $right.Id
| project TimeGenerated, OperationId, Id, ParentId, AppRoleName, Name, Kind
| order by TimeGenerated desc
```

Interpretation: this returns candidates, not proof of defects. A missing parent may be a remote/broker/platform span, outside the query window, in another resource/workspace, sampled out, or stored under a different telemetry type. Certification should compare against the expected boundary map.

19. Verification and certification procedure

Step	Required evidence
Inventory freeze	Record versions of browser SDK, APIM SKU/API/configuration, App Service JDK image, Java agent, frameworks/clients, messaging SDKs, Functions runtime/worker and Azure resources.
100% fidelity	Apply bounded 100% certification sampling at every producer. Confirm diagnostics are active and body capture remains disabled.
Synchronous test	Run a synthetic browser → APIM → service A → service B → PostgreSQL + Redis transaction with a known business ID.
ID invariant A	All trace telemetry rows for the transaction share exactly one expected W3C trace-id / AI OperationId unless an intentional trust-boundary break is documented.
ID invariant B	Every real span has a non-zero unique span-id. Every strict child ParentId equals the actual parent span Id.
ID invariant C	Browser dependency → APIM gateway → Java server/client spans form the expected graph; no manufactured APIM request ID appears as trace-id.
Sampling invariant	A `-00` inbound trace is not rewritten to `-01`. Local recording behavior is documented independently.
Logging invariant	Structured logs contain the active trace-id/span-id and business ID without credentials, tokens, raw claims, query parameters, cache keys or bodies.
Async retry test	Inject a transient consumer failure; verify separate attempt spans, preserved message/business identity, and visible attempt count.
Dead-letter/replay test	Force dead-letter and replay; verify business ID continuity and honest link/new-trace semantics.
Fan-out/fan-in test	Generate multiple branch consumers and one aggregation; verify multiple links for fan-in, not fabricated parentage.
Batch test	Create a mixed-context batch; verify links or bounded per-item representation and no arbitrary first-message parent.
Data-tier test	Verify JDBC and Redis client dependencies, safe attributes and no claim of server-native continuation.
Visualization	Export Application Map and Transaction Diagnostics evidence; confirm role names and edges match the reference architecture.
KQL evidence	Save trace reconstruction, business lookup and orphan-candidate query results with timestamps and configurations.
Load and failure test	Measure latency, CPU, memory, throughput, ingestion volume, exporter backpressure and restart behavior with diagnostics enabled.
Sign-off	Architecture, application, platform, SRE/observability, security/privacy and data owners approve evidence and residual limitations.

20. Failure modes and limitations checklist

☐ Browser SDK loads after early calls; initial AJAX dependency has no context.

☐ CORS rejects trace headers or SDK exclusion list suppresses expected propagation.

☐ Correlation headers are sent to an untrusted third-party origin.

☐ APIM diagnostic exists at the wrong scope or references the wrong logger/resource.

☐ APIM is set to legacy correlation instead of W3C.

☐ APIM and Java sampling rates differ, creating partial traces.

☐ Policy rewrites `-00` to `-01` or fabricates context from `context.RequestId`.

☐ Excessive header/body logging degrades gateway throughput or leaks data.

☐ Java agent is loaded twice or another framework installs a second provider.

☐ JDK 25 bytecode/instrumentation incompatibility silently disables an instrumentation module.

☐ Service names collide across services/environments; Application Map merges unrelated nodes.

☐ App/worker shuts down before batch exporter flushes.

☐ Log framework loses MDC across thread pools/reactive context.

☐ HTTP framework uses nonstandard carrier or strips `tracestate`.

☐ Messaging SDK instrumentation is absent for the selected Java version.

☐ Queue envelope truncates or corrupts context; invalid input is accepted without validation.

☐ Consumer reuses span ID across deliveries/retries.

☐ Replay falsely continues an ancient parent as though the span were still active.

☐ Fan-in uses one input as parent and loses the other causal contributors.

☐ Batch link count is unbounded and creates telemetry explosions.

☐ Dead-letter tooling drops application properties or business ID.

☐ CloudEvent `id`, message ID, APIM request ID or Entra correlation ID is mistaken for W3C trace-id.

☐ JDBC statement capture leaks literals, personal data or credentials.

☐ Instrumentation shows only connection-pool operations, not logical queries.

☐ Redis keys/values/scripts are collected, creating privacy and cardinality risk.

☐ Existing Azure Cache for Redis retirement deadlines are omitted from the platform roadmap.

☐ Business invocation ID is caller-controlled, predictable, or used as an authorization secret.

☐ Business ID becomes a metric dimension, causing cardinality/cost problems.

☐ Workspace RBAC grants broader access than the separate AI-resource design intended.

☐ Cross-resource query omits one Application Insights resource; “orphan” is a query-scope artifact.

☐ Clock skew or ingestion delay misleads time-order analysis.

☐ Retention, workspace transformation or ingestion limit removes evidence.

☐ Application Map appears correct but a sampled critical path is missing.

☐ Collector tail sampling drops late spans, exhausts memory or becomes a single point of telemetry failure.

☐ Preview Trace Analytics/native OTLP path changes schema or availability.

☐ Third-party exporter duplicates, re-batches or changes resource attributes.

☐ Certification used happy-path traffic only and failed to cover retries, replay, fan-in and partial outage.

21. Retain / amend / remove from earlier guidance

Earlier recommendation	Disposition	Corrected form
W3C everywhere	Retain	W3C is the sole technical causality protocol; validate at trust boundaries.

Application Insights as preferred backend	Retain	Use workspace-based AI/LAW; preserve OTel portability.
Application Insights Java agent	Retain with gate	Pin and certify against JDK 25; one provider per JVM.
Business correlation header	Amend	Use only as durable business invocation ID; trusted-edge validation; separate from trace-id.
APIM W3C diagnostics	Retain	Set `httpCorrelationProtocol=W3C`; test emitted parentage.
Rewrite sampled bit	Remove	Never mutate `-00` to `-01`.
Mint traceparent from APIM request ID	Remove	Use standards tracer/root creation; request ID remains a separate gateway diagnostic attribute.
APIM native OTLP	Remove unless newly documented	Current design relies on documented AI diagnostics.
send-request/wait guarantees spans	Amend	Validate actual gateway telemetry; instrument app calls where required.
.NET Service Bus 7.13.0 feature flag	Remove	Irrelevant to Java.
Classic adaptive sampling for Java	Remove/amend	Use pinned Java agent/OTel sampler documentation.
One AI resource globally	Amend	One per workload/environment default; shared LAW with governance exceptions.
Azure Cache for Redis for new deployments	Remove/outdated	Use Azure Managed Redis; migrate existing estates before retirement.
100% all the time	Amend	100% for bounded certification/forensic mode; measured production strategy.
Collector for every workload	Amend	Optional when tail sampling, multi-backend or routing requirements justify it.

22. Atomic action plan

Phase A — Architecture decisions

ID	Owner role	Atomic action	Evidence of completion	Prerequisite
A01	Architecture	Approve W3C Trace Context as the only technical causality protocol.	Signed ADR naming traceparent/tracestate and span links.	None
A02	Architecture	Approve `business.invocation_id` as a separate optional durable identifier.	ADR defines use cases, format, trust boundary, retention and privacy.	A01
A03	Architecture/ Security	Define external trace-context trust policy.	Rules for valid/invalid/oversized context and boundary breaks.	A01
A04	Observability	Choose direct Azure Monitor export as baseline; record Collector triggers.	ADR compares direct, Collector and third-party options.	A01
A05	Platform governance	Approve AI resource/workspace topology.	Diagram and IaC naming/RBAC/retention map.	A01
A06	Data architecture	Select Azure Managed Redis for new deployment.	Decision cites retirement dates and target SKU/region.	None

Phase B — Platform provisioning

ID	Owner role	Atomic action	Evidence of completion	Prerequisite
B01	Platform	Create workspace-based Application Insights resources per workload/environment.	Resource IDs and deployment output.	A05
B02	Platform	Create or associate governed Log Analytics workspace(s).	Workspace retention, RBAC, CMK/data boundary decisions captured.	A05
B03	Platform	Create APIM Application Insights logger through IaC.	Deployed logger references correct AI resource; secret not in source.	B01
B04	Platform	Create APIM diagnostic with W3C correlation.	Exported resource shows W3C, sampling and allowlists.	B03
B05	Platform/ FinOps	Create budgets/alerts for telemetry ingestion.	Budget and ingestion-volume workbook/alert.	B01-B02
B06	Platform/ Security	Create resource-context RBAC assignments.	Access test shows least privilege by role.	B01-B02
B07	Platform	Provision Azure Managed Redis and private connectivity.	Connection smoke test; no secrets in logs.	A06

Phase C — Browser and front end

ID	Owner role	Atomic action	Evidence of completion	Prerequisite
C01	Web engineering	Inventory SPA and server-rendered request paths.	Boundary map lists every API origin and SSR hop.	A01
C02	Web engineering	Pin Application Insights JavaScript SDK version.	Lockfile/SBOM and approved version record.	C01
C03	Web engineering	Enable W3C distributed tracing mode.	Browser network capture shows valid traceparent.	C02
C04	Web/Security	Configure correlation allow/exclusion domains.	Automated test proves headers only reach approved origins.	C03
C05	Web/Platform	Update CORS for required headers.	Preflight succeeds for approved APIM origin only.	C04
C06	Privacy/Web	Implement pseudonymous user/session enrichment.	Threat/privacy review; no raw identity in telemetry.	A02

C07	Web engineering	Instrument SPA route/page semantics.	Page views and AJAX dependencies appear with expected context.	C03
C08	Web engineering	Instrument server-rendered App Service independently.	SSR requests/dependencies appear under stable role name.	B01

Phase D — APIM

ID	Owner role	Atomic action	Evidence of completion	Prerequisite
D01	API platform	Set diagnostic sampling to 100% in certification environment.	Configuration export and emitted gateway telemetry.	B04
D02	API platform	Set `alwaysLog` for supported error behavior.	Failure test shows expected error telemetry; semantics documented.	B04
D03	API/ Security	Allowlist business ID header; redact forbidden headers.	Policy test shows no Authorization/cookie/token capture.	A02,B04
D04	API platform	Set request/response body capture to zero by default.	Diagnostic export confirms body bytes=0.	B04
D05	API platform	Remove trace flag rewrite logic.	Policy repository scan finds no `-00→-01` rewrite.	A03
D06	API platform	Remove traceparent-from-`context.RequestId` logic.	Policy and runtime capture prove no synthetic conversion.	A03
D07	API platform	Test inbound valid, absent and invalid traceparent cases.	Three evidence captures and expected outcomes.	D01
D08	SRE/API	Load test diagnostics and policy overhead.	Before/after latency, CPU, capacity and telemetry volume report.	D01-D04

Phase E — Java services

ID	Owner role	Atomic action	Evidence of completion	Prerequisite
E01	Java engineering	Inventory every JDK 25 service, framework and client library.	SBOM with HTTP/JDBC/Redis/messaging libraries.	None
E02	Java/ Observability	Select and pin Azure Monitor Java agent version.	Approved binary checksum and release/support evidence.	E01
E03	Java/Platform	Attach agent exactly once in App Service startup.	Startup logs show one agent/provider and successful exporter.	E02
E04	Java engineering	Set stable `service.name`/role name per service/ environment.	Application Map shows distinct deterministic nodes.	E03
E05	Java engineering	Enable structured logging correlation.	Log row trace/span IDs match active span.	E03
E06	Java engineering	Remove duplicate OTel/Application Insights SDK providers/exporters.	Dependency and runtime scan shows one provider.	E03
E07	Java engineering	Add manual spans only for uncovered business boundaries.	Code review and trace show non-duplicated custom spans.	E06
E08	Java/SRE	Run JDK 25 instrumentation compatibility suite.	HTTP, JDBC, Redis, logging, messaging, shutdown and restart results.	E03-E07
E09	Java/Security	Validate inbound business ID at the trusted service edge.	Property-based tests for length/charset/trust cases.	A02
E10	Java engineering	Propagate business ID on approved HTTP clients.	Integration test shows continuity without using trace-id.	E09

Phase F — PostgreSQL and Redis

ID	Owner role	Atomic action	Evidence of completion	Prerequisite
F01	Java/Data	Verify JDBC automatic instrumentation for the exact driver/ framework.	Dependency span includes safe DB attributes and expected parent.	E08
F02	Security/ Data	Disable/sanitize SQL text and parameter capture.	Telemetry review contains no literals/credentials/PII.	F01
F03	Data/SRE	Document client-span versus server-diagnostic boundary.	Runbook states limits and time-correlation method.	F01
F04	Java/Data	Verify exact Redis client instrumentation.	CLIENT spans appear once with safe operation attributes.	E08,B07
F05	Security/ Data	Block raw Redis keys/values/scripts/arguments.	Telemetry inspection and tests pass.	F04
F06	Data/ Platform	Create migration plan for any existing Azure Cache for Redis.	Inventory, target, test, cutover, rollback and deadlines approved.	A06

Phase G — Async messaging and workers

ID	Owner role	Atomic action	Evidence of completion	Prerequisite
G01	Integration architecture	Define versioned message context envelope/ property schema.	Schema includes W3C context, business ID, message identity and size limits.	A01-A03
G02	Java/Integration	Instrument Service Bus send and process paths.	Producer/consumer trace with retry attempt evidence.	G01,E08
G03	Java/Integration	Instrument Event Grid CloudEvents path.	CloudEvent ID remains separate; context survives delivery.	G01
G04	Java/Integration	Instrument Event Hubs send/receive/batch path.	Batch test uses links or documented bounded representation.	G01
G05	Java/Integration	Instrument Storage Queue path.	Message schema round-trips within size and privacy limits.	G01
G06	Functions team	Validate Java Functions/worker trigger propagation.	No duplicate host/worker spans; process parentage verified.	G01
G07	Integration/SRE	Implement retry attempt semantics.	Each delivery has a new span ID and recorded attempt count.	G02-G06
G08	Integration/SRE	Implement dead-letter preservation and replay policy.	DLQ/replay test preserves safe identity and uses link/new trace.	G07
G09	Integration architecture	Implement fan-out and fan-in span relationships.	Fan-out children and fan-in links visible in exported telemetry.	G02-G06
G10	Security/Integration	Validate untrusted message context.	Malformed/spoofed context test follows trust policy.	A03,G01

Phase H — Identity, privacy and governance

ID	Owner role	Atomic action	Evidence of completion	Prerequisite
H01	Identity/Security	Define safe Entra telemetry attribute allowlist.	Policy excludes tokens/raw claims and separates Entra IDs from trace IDs.	A03
H02	Privacy	Approve pseudonymization key management and rotation.	HMAC design, access and retention approved.	C06
H03	Privacy/ Observability	Run telemetry data classification scan.	No forbidden fields in AI/LAW samples.	B01-B04,E05,F02,F05
H04	Governance	Set retention, purge and export rules for business IDs.	Configured retention and tested deletion/purge procedure.	A02,B02
H05	Security	Restrict telemetry query/export access.	RBAC access test and audit evidence.	B06

Phase I — Sampling and validation

ID	Owner role	Atomic action	Evidence of completion	Prerequisite
I01	Observability	Document all sampling decision points and defaults.	Sampling matrix per browser/APIM/JVM/Collector/backend.	C03,D01,E03
I02	Observability	Enable bounded 100% certification mode.	Change record and configuration snapshots.	I01
I03	Test engineering	Execute synchronous invariant test.	Saved headers, rows, graph and KQL output.	I02,F01,F04
I04	Test engineering	Execute error/timeout test.	Errors retain trace/business correlation.	I02
I05	Test engineering	Execute retry/DLQ/replay tests.	Attempt spans and links/new trace semantics pass.	G07-G08,I02
I06	Test engineering	Execute fan-out/fan-in/batch tests.	Links and graph pass expected model.	G09,I02
I07	Observability	Run trace reconstruction and business lookup KQL.	Queries saved with reproducible results.	I03-I06
I08	Observability	Run orphan-candidate query and triage every class.	Each candidate classified: expected remote, cross-resource, sampled, delayed, or defect.	I07
I09	SRE/FinOps	Measure telemetry overhead and cost at representative load.	Performance/cost report with thresholds.	I02

Phase J — Production rollout and preview evaluation

ID	Owner role	Atomic action	Evidence of completion	Prerequisite
J01	Architecture board	Approve certification evidence and residual limitations.	Signed acceptance against section 24.	I03-I09
J02	SRE/Observability	Set production sampling strategy.	Version-controlled ratios/rates/error rules and rollback.	J01
J03	Release engineering	Canary rollout browser/APIM/first Java service.	Canary trace completeness and SLOs pass.	J02
J04	Release engineering	Roll out remaining synchronous services.	Application Map/queries show expected topology.	J03
J05	Release engineering	Roll out messaging and workers.	Async certification vectors pass in production canary.	J04
J06	SRE	Enable alerts for telemetry loss and exporter failures.	Synthetic trace monitor and alert action evidence.	J03-J05
J07	Platform	Evaluate Collector only against documented trigger.	POC proves tail sampling/multi-route need, HA and capacity or is rejected.	A04,J01
J08	Platform	Evaluate Azure Monitor Traces / Trace Analytics preview in isolation.	Preview POC, terms/region/schema/rollback and no production dependency.	J01
J09	Platform	Re-check GA/preview status quarterly.	Dated status register with official links.	J01
J10	Operations	Schedule periodic correlation regression suite.	Automated synthetic tests and evidence retention.	J05-J06

23. Production-ready checklist

☐ W3C is configured and verified across browser, APIM, Java services and async carriers.

☐ No policy rewrites sampled flags or fabricates traceparent from APIM request ID.

☐ Exactly one OpenTelemetry provider runs per Java JVM; agent/JDK 25 compatibility is evidenced.

☐ Stable service/role names produce an accurate Application Map.

☐ JDBC and Redis dependency spans are present and contain no sensitive statements/keys/values.

☐ Azure Managed Redis is the new-architecture target; existing Azure Cache for Redis has a dated migration plan.

☐ Entra request/correlation IDs remain separate from W3C IDs; identity enrichment is pseudonymous and minimal.

☐ `business.invocation_id` is validated at a trusted edge, propagated safely and excluded from metrics.

☐ Async retry, dead-letter, replay, fan-out, fan-in and batch paths have verified semantics.

☐ APIM/browser/Java sampling independence is documented; 100% certification mode can be enabled safely.

☐ AI/LAW topology, RBAC, retention, purge, budgets and alerting are configured.

☐ The three KQL procedures execute against the current workspace schema.

☐ Telemetry loss/exporter failure has a synthetic detector and alert route.

☐ Preview capabilities are isolated from the production dependency chain.

☐ No secrets, tokens, raw claims, payload bodies, SQL literals or cache data are present in telemetry evidence.

24. Acceptance criteria: “end-to-end correlation certified”

Criterion	Pass condition
Coverage	Every declared synchronous boundary and each supported async transport emitted the expected spans/links under controlled 100% fidelity.
Trace identity	All rows for a synchronous test chain share the expected non-zero W3C trace-id/OperationId, except documented trust-boundary/new-trace cases.
Span identity	Every emitted span has a unique non-zero span-id; strict child ParentId matches the actual parent Id.
No falsification	No sampled-flag rewrite, APIM request-ID conversion or Entra-ID conversion occurs.
Async honesty	Retries are separate attempts; replay is linked or a new trace; fan-in uses links; batches do not invent a single false parent.
Durable lookup	One known business invocation ID returns all intended traces and only the approved telemetry attributes.
Data safety	Automated/manual review finds no credentials, access tokens, authorization headers, raw personal claims, request/response bodies, SQL literals, Redis keys/values or business payloads.
Operational stability	Representative load shows telemetry overhead within approved SLO/capacity/cost thresholds; restart/shutdown does not produce unacceptable loss.
Backend usability	Application Map, Transaction Diagnostics and KQL reconstruct the expected graph and role names.
Governance	Owners approve resource topology, RBAC, retention, purge, budget, sampling and preview exclusions.
Repeatability	The test suite and IaC can recreate the result after deployment, runtime/agent upgrade or message-client change.

> **Certification statement**
> Declare certification only for the named environment, versions, paths and sampling configuration in the evidence pack. A dependency/runtime/agent/policy/message-schema change invalidates the affected portion until the regression suite passes.

25. Source and reference register

Material standards and product claims should be rechecked at implementation time. URLs below are intentionally explicit and clickable. Official Microsoft documentation is prioritized; W3C and OpenTelemetry specifications are the normative sources for trace semantics.

[S1] W3C Trace Context Recommendation — Normative standard. https://www.w3.org/TR/trace-context/

[S2] OpenTelemetry Trace API and SDK specification — Normative standard. https://opentelemetry.io/docs/specs/otel/trace/

[S3] Telemetry correlation in Application Insights — Official Microsoft Learn. https://learn.microsoft.com/azure/azure-monitor/app/distributed-trace-data

[S4] Integrate Azure API Management with Application Insights — Official Microsoft Learn. https://learn.microsoft.com/azure/api-management/api-management-howto-app-insights

[S5] API Management diagnostic resource schema — Official Microsoft Learn / ARM schema. https://learn.microsoft.com/azure/templates/microsoft.apimanagement/service/apis/diagnostics

[S6] API Management capacity and pricing caveats — Official Microsoft Azure. https://azure.microsoft.com/pricing/details/api-management/

[S7] Application Insights Java agent / Java 3.x configuration — Official Microsoft Learn. https://learn.microsoft.com/azure/azure-monitor/app/java-in-process-agent

[S8] Configure Java apps in Azure App Service — Official Microsoft Learn. https://learn.microsoft.com/azure/app-service/configure-language-java-deploy-run

[S9] Application Insights Java configuration options — Official Microsoft Learn. https://learn.microsoft.com/azure/azure-monitor/app/java-standalone-config

[S10] Sampling in Azure Monitor Application Insights — Official Microsoft Learn. https://learn.microsoft.com/azure/azure-monitor/app/sampling

[S11] OpenTelemetry Java automatic instrumentation — Official OpenTelemetry. https://opentelemetry.io/docs/zero-code/java/agent/

[S12] Application Insights JavaScript SDK configuration — Official Microsoft Learn. https://learn.microsoft.com/azure/azure-monitor/app/javascript-sdk-configuration

[S13] Application Insights JavaScript SDK repository — Official Microsoft GitHub. https://github.com/microsoft/ApplicationInsights-JS

[S14] Azure SDK for Java tracing with OpenTelemetry — Official Microsoft Learn. https://learn.microsoft.com/azure/developer/java/sdk/tracing

[S15] OpenTelemetry messaging semantic conventions — Normative OpenTelemetry. https://opentelemetry.io/docs/specs/semconv/messaging/

[S16] CloudEvents distributed tracing extension — Official CloudEvents specification. https://github.com/cloudevents/spec/blob/main/cloudevents/extensions/distributed-tracing.md

[S17] Azure Event Hubs for Java — Official Microsoft Learn. https://learn.microsoft.com/java/api/overview/azure/messaging-eventhubs-readme

[S18] Azure Queue Storage for Java — Official Microsoft Learn. https://learn.microsoft.com/java/api/overview/azure/storage-queue-readme

[S19] Monitor Azure Functions with OpenTelemetry distributed tracing — Official Microsoft Learn. https://learn.microsoft.com/azure/azure-functions/monitor-functions-opentelemetry-distributed-tracing

[S20] Use Java and JDBC with Azure Database for PostgreSQL — Official Microsoft Learn. https://learn.microsoft.com/azure/postgresql/connectivity/connect-java

[S21] Dependency tracking in Application Insights — Official Microsoft Learn. https://learn.microsoft.com/azure/azure-monitor/app/dependencies

[S22] Azure Cache for Redis retirement and Azure Managed Redis direction — Official Microsoft Azure; verified current 31 Oct 2026. https://azure.microsoft.com/products/cache

[S23] Troubleshoot Microsoft Entra sign-ins using request/correlation IDs — Official Microsoft Learn. https://learn.microsoft.com/entra/identity/monitoring-health/troubleshoot-sign-in-with-request-id

[S24] Azure Monitor OpenTelemetry overview — Official Microsoft Learn. https://learn.microsoft.com/azure/azure-monitor/app/opentelemetry-overview

[S25] Azure Monitor Traces / Trace Analytics documentation — Official Microsoft Learn; status must be checked. https://learn.microsoft.com/azure/azure-monitor/app/trace-analytics

[S26] Application Map in Application Insights — Official Microsoft Learn. https://learn.microsoft.com/azure/azure-monitor/app/app-map

[S27] Application Insights end-to-end transaction diagnostics — Official Microsoft Learn. https://learn.microsoft.com/azure/azure-monitor/app/transaction-search-and-diagnostics

[S28] Workspace-based Application Insights resources — Official Microsoft Learn. https://learn.microsoft.com/azure/azure-monitor/app/create-workspace-resource

[S29] Application Insights tables in Azure Monitor Logs — Official Microsoft Learn. https://learn.microsoft.com/azure/azure-monitor/reference/tables-category

[S30] Preview supplemental terms — Official Microsoft Azure. https://azure.microsoft.com/support/legal/preview-supplemental-terms/

[S31] OpenTelemetry Collector documentation — Official OpenTelemetry. https://opentelemetry.io/docs/collector/

[S32] OpenTelemetry span links concept — Official OpenTelemetry. https://opentelemetry.io/docs/concepts/signals/traces/#span-links

[S33] Azure Monitor data platform and distributed traces — Official Microsoft Learn. https://learn.microsoft.com/azure/azure-monitor/fundamentals/data-platform

[S34] Azure messaging services documentation hub — Official Microsoft Learn. https://learn.microsoft.com/azure/messaging-services/

[S35] Azure Event Grid SDK for Java — Official Microsoft Learn. https://learn.microsoft.com/java/api/overview/azure/event-grid

26. Final architecture position

The defensible architecture is a standards-first trace graph, not a universal correlation string. W3C context provides technical causality; OpenTelemetry defines span and link semantics; APIM is configured as a W3C-aware gateway telemetry boundary; each Java JDK 25 process has one validated provider; JDBC and Redis produce safe client dependencies; messaging preserves context and uses links for non-tree causality; Entra identity IDs remain separate; and a durable business invocation ID is added only to bridge traces across long-running/replayed business operations. Azure Monitor Application Insights is the production system of record, with preview OTLP/Trace Analytics paths isolated until the exact paths are GA and operationally accepted.

This position retains the valuable core of the original recommendation while explicitly rejecting the disputed shortcuts: sample-flag rewriting, traceparent fabrication from APIM request IDs, unverified APIM native OTLP assumptions, .NET-only Java guidance, classic adaptive-sampling claims, and continued use of a retiring cache service for a new architecture.